EXPERIMENT 6 – Check Sum Validation

Name: Muthu Venkatesh M

Reg No: 24BCE1389

Github Link: https://github.com/muthuvenkatesh-24/network-traffic-analysis

Method 1:

IPv4

```
[Stream index: 0]
▾ Internet Protocol Version 4, Src: 142.250.66.14, Dst: 172.16.15.114
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x60 (DSCP: CS3, ECN: Not-ECT)
    Total Length: 60
    Identification: 0x0000 (0)
  ▸ 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 115
    Protocol: ICMP (1)
    Header Checksum: 0xbad6 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 142.250.66.14
    Destination Address: 172.16.15.114
    [Stream index: 216]
```

TCP:

```
▾ Transmission Control Protocol, Src Port: 58525, Dst Port: 7680, Seq: 0, Len: 0
    Source Port: 58525
    Destination Port: 7680
    [Stream index: 2]
    [Stream Packet Number: 2]
  ▸ [Conversation completeness: Incomplete, SYN_SENT (1)]
    [TCP Segment Len: 0]
    Sequence Number: 0    (relative sequence number)
    Sequence Number (raw): 2754402574
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 0
    Acknowledgment number (raw): 0
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x002 (SYN)
    Window: 65535
    [Calculated window size: 65535]
    Checksum: 0x3b71 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted
  ▸ [Timestamps]
  ▸ [SEQ/ACK analysis]
    [Client Contiguous Streams: 0]
    [Server Contiguous Streams: 1]
```

UDP:

```
> Frame 2526: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on Interface \Device\NPF_{AC10383A-0643-473A-8F90-EF369A9AE682}, id 0
> Ethernet II, Src: HonHaiPrecis_f8:10:84 (d0:f4:05:f8:10:84), Dst: Cisco_0f:22:48 (58:56:9f:0f:22:48)
> Internet Protocol Version 4, Src: 172.16.15.114, Dst: 172.16.1.11
v User Datagram Protocol, Src Port: 65478, Dst Port: 53
    Source Port: 65478
    Destination Port: 53
    Length: 52
    Checksum: 0x68e3 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 50]
    [Stream Packet Number: 1]
  > [Timestamps]
    UDP payload (44 bytes)
> Domain Name System (query)
```

TLS:

```
> Frame 2513: Packet, 116 bytes on wire (928 bits), 116 bytes captured (928 bits) on Interface \Device\NPF_{AC10383A-0643-473A-8F90-EF369A9AE682}, id 0
> Ethernet II, Src: Cisco_0f:22:48 (58:56:9f:0f:22:48), Dst: HonHaiPrecis_f8:10:84 (d0:f4:05:f8:10:84)
v Internet Protocol Version 4, Src: 4.150.240.254, Dst: 172.16.15.114
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 102
    Identification: 0x7b30 (31536)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 63
    Protocol: TCP (6)
    Header Checksum: 0x0f4b [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 4.150.240.254
    Destination Address: 172.16.15.114
    [Stream index: 209]
> Transmission Control Protocol, Src Port: 443, Dst Port: 33993, Seq: 4884, Ack: 3273, Len: 62
v Transport Layer Security
    [Stream index: 5]
  v TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
      Opaque Type: Application Data (23)
      Version: TLS 1.2 (0x0303)
      Length: 57
      Encrypted Application Data: 2d939cec00040f0f096032761bf3dcab33412ba46560aa05a4b43e316d7babcc69fffc9ecf9d0ddb6450ba464d5f76e6b7f72f16bc934fec33
      [Application Data Protocol: Hypertext Transfer Protocol]
```

Ethernet:

```
> Frame 1: Packet, 121 bytes on wire (968 bits), 121 bytes captured (968 bits) on Interface \Device\NPF_{AC10383A-0643-473A-8F90-EF369A9AE682}, id 0
v Ethernet II, Src: Cisco_0f:22:48 (58:56:9f:0f:22:48), Dst: HonHaiPrecis_f8:10:84 (d0:f4:05:f8:10:84)
  v Destination: HonHaiPrecis_f8:10:84 (d0:f4:05:f8:10:84)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  v Source: Cisco_0f:22:48 (58:56:9f:0f:22:48)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 0]
> Internet Protocol Version 4, Src: 172.64.155.200, Dst: 172.16.15.114
> User Datagram Protocol, Src Port: 443, Dst Port: 60201
> Data (79 bytes)
```

Method 2:
Take TCP

M. Muthu Venkatesh
24BCE1389

Manual calculation for checksum (TCP)

Source IP → 172.16.76.34

Destination IP → 172.16.15.131

Reserved → 0

Protocol → 6

TCP length → 32 bytes

1's complement addition:

AC10 + 4C221 + AC10 + DF83 + 0006 + 0020

= 10C8B

⇒ 0C8B + 1 = $\boxed{0C8C}$

Now, add TCP header

0C8C + E49D + 1E00 + A42C + D40E + 0000 + 0000 + 8002 + FFFF + 0000 + 0204 + 05B4 + 0103 + 0308 + 0101 + 0402 = $\boxed{C48E}$

1's complement of final sum,

FFFF - C48E = $\boxed{3B71}$

Calculated TCP checksum = 0X3B71

Wireshark checksum = 0X3B71

∴ Matches, verified manually.

Method 3:
Automation using scapy

Code:

```
from scapy.all import *

pcap_file = "checksum_capture.pcapng"
packets = rdpcap(pcap_file)

def validate_ip_checksum(pkt):
    if IP in pkt:
        original = pkt[IP].chksum
        del pkt[IP].chksum
        recalculated = IP(bytes(pkt[IP])).chksum
        return original == recalculated
    return None

def validate_tcp_checksum(pkt):
    if IP in pkt and TCP in pkt:
        original = pkt[TCP].chksum
        del pkt[TCP].chksum
        recalculated = TCP(bytes(pkt[TCP])).chksum
        return original == recalculated
    return None

def validate_udp_checksum(pkt):
    if IP in pkt and UDP in pkt:
        original = pkt[UDP].chksum
        del pkt[UDP].chksum
        recalculated = UDP(bytes(pkt[UDP])).chksum
        return original == recalculated
```

```
    return None

print("CHECKSUM VALIDATION USING
SCAPY") print("--------------------------------")

for i, pkt in enumerate(packets, start=1):
    if IP in pkt:
        ip_ok = validate_ip_checksum(pkt)
        tcp_ok = validate_tcp_checksum(pkt)
udp_ok = validate_udp_checksum(pkt)

print(f"Packet {i}:")
if ip_ok is not None:
print(f" IP Checksum Valid : {ip_ok}")
if tcp_ok is not None:
print(f" TCP Checksum Valid : {tcp_ok}")
if udp_ok is not None:
print(f" UDP Checksum Valid : {udp_ok}")
```

O/P:

```
C:\Users\Student\Documents>python checksum_validation.py
CHECKSUM VALIDATION USING SCAPY
------------------------------
Packet 1:
  IP Checksum Valid  : True
  UDP Checksum Valid : True
Packet 2:
  IP Checksum Valid  : False
  UDP Checksum Valid : False
Packet 3:
  IP Checksum Valid  : True
  TCP Checksum Valid : True
Packet 5:
  IP Checksum Valid  : True
  TCP Checksum Valid : True
Packet 6:
  IP Checksum Valid  : True
  TCP Checksum Valid : True
Packet 7:
  IP Checksum Valid  : True
  TCP Checksum Valid : True
Packet 8:
  IP Checksum Valid  : True
  UDP Checksum Valid : True
Packet 9:
  IP Checksum Valid  : True
  TCP Checksum Valid : True
Packet 10:
  IP Checksum Valid  : False
  UDP Checksum Valid : False
Packet 11:
  IP Checksum Valid  : True
  UDP Checksum Valid : True
Packet 12:
  IP Checksum Valid  : False
  UDP Checksum Valid : False
Packet 13:
  IP Checksum Valid  : True
  UDP Checksum Valid : True
Packet 15:
  IP Checksum Valid  : True
  TCP Checksum Valid : True
```